



同濟大學
TONGJI UNIVERSITY

2025-2026 学年第二学期

基于 STM32 与 X-CUBE-AI 的手 写数字识别系统设计

姓 名：赵昕柔，余世瑶

学 号：2452491，2451274

学 院：电子与信息工程学院

课 程：嵌入式系统(10024603)

指导教师：徐和根

一、项目背景

随着嵌入式人工智能技术的发展，神经网络模型已经不再局限于电脑端或云端运行。通过模型压缩、代码自动生成和嵌入式部署，可以将训练完成的神经网络移植到微控制器中，实现低功耗、低延迟的本地推理。

本课题基于 STM32 微控制器和 X-CUBE-AI 工具设计并实现了一个手写数字识别系统。用户可以在电脑端界面中书写数字，程序将图像转换为固定尺寸的数据并通过串口发送至 STM32 开发板。开发板接收到图像数据后调用部署在芯片中的神经网络模型完成分类，并将识别结果及各类别概率返回电脑端显示。

该系统展示了从模型生成、嵌入式部署、串口通信到图形化交互的完整流程，具有较强的综合实践意义。

二、项目设计目标

本项目旨在基于 STM32F767 开发板，利用 STM32Cube.AI 工具实现手写数字识别模型的嵌入式部署并构建一个完整的手写数字识别系统。系统通过上位机手写输入数字图像，并经串口传输至开发板；开发板完成图像数据接收、神经网络推理计算以及识别结果生成，并将预测结果及各类别概率信息返回上位机进行显示。具体而言主要实现了以下内容：

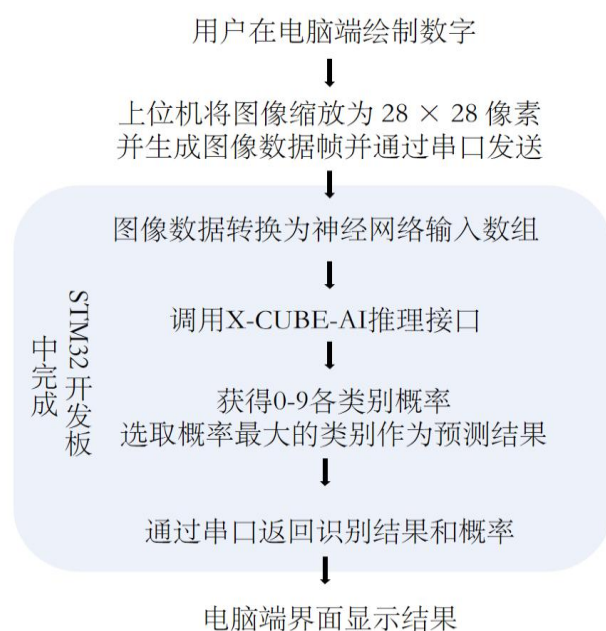
1. 在 STM32 开发板微控制器中部署手写数字分类神经网络；
2. 实现电脑端与 STM32 开发板之间的串口通信及数据传输；
3. 将手写图像转换为神经网络可以处理的输入格式，在 STM32 中完成本地推理并向电脑端返回识别结果；
4. 在电脑端显示手写数字的实时识别及对应预测概率；
5. 分析调试过程中出现的串口通信问题并完成优化。

三、系统总体设计

系统主要由电脑端上位机程序和 STM32 嵌入式程序两部分组成。

3.1 系统架构

系统的整体工作流程如下图所示：



3.2 系统组成

模块	功能
上位机绘图模块	提供手写区域，采集用户书写的数字
图像处理模块	将手写图像转换为 28×28 像素数据
串口发送模块	将图像数据发送至 STM32
STM32 串口接收模块	接收图像数据并判断一帧数据是否完整
AI 推理模块	调用部署后的神经网络模型完成分类
串口返回模块	返回识别数字及每个类别对应的概率
上位机显示模块	在界面中展示识别结果

四、项目平台

4.1 硬件平台

本项目使用 STM32F767 系列微控制器作为核心控制器。该芯片基于 ARM Cortex-M7 内核，具有较高的主频和较强的浮点计算能力，可以满足小型神经网络模型的推理需求。

电脑与开发板之间通过串口进行通信，串口波特率设置为 115200bit/s。

4.2 软件平台

软件或工具	主要用途
STM32CubeIDE	STM32 工程开发、编译与烧录
STM32CubeMX	外设配置与初始化代码生成
X-CUBE-AI	神经网络模型转换与嵌入式部署

C 语言	STM32 端程序开发
C# Windows Forms	电脑端手写数字界面与串口通信

五、关键模块实现

5.1 神经网络部署

本项目采用开源仓库提供的预训练手写数字识别模型 `trained_lstm.tflite` 作为神经网络模型，并通过 STM32Cube.AI 完成模型解析、代码生成以及嵌入式部署。

5.1.1 输入与输出

模型输入为 28×28 像素的图像，共包含 784 个像素值；模型输出为对应数字 0-9 的预测概率的 10 个浮点数及最终预测结果。

5.1.2 Cube.AI 初始化

STM32 程序启动后首先需要创建并初始化神经网络实例：

```
static void AI_Init(void)
{
    ai_error err;

    const ai_handle act_addr[] = { activations };

    err = ai_network_create_and_init(&network, act_addr, NULL);

    if (err.type != AI_ERROR_NONE)
    {
        Error_Handler();
    }

    ai_input = ai_network_inputs_get(network, NULL);
    ai_output = ai_network_outputs_get(network, NULL);
}
```

其中，`activations` 数组用于保存神经网络运行过程中的中间变量。

5.1.3 图像数据转换

电脑端将手写图像转换为字节数组。STM32 接收数据后，需要将图像像素转换为浮点型数组，作为神经网络输入。

```

void PictureCharArrayToFloat(uint8_t *srcBuf,
                             float *dstBuf,
                             int len)
{
    for (int i = 0; i < len; i++)
    {
        dstBuf[i] = srcBuf[i];
    }
}

```

当前程序直接将每个像素值转换为浮点数，不进行额外归一化处理。该处理方式需要与模型训练阶段使用的输入格式保持一致。

5.1.4 AI 推理过程

STM32 收到图像数据后，将输入和输出数组地址传给神经网络，并调用推理接口：

```

ai_input[0].data = AI_HANDLE_PTR(pIn);
ai_output[0].data = AI_HANDLE_PTR(pOut);

batch = ai_network_run(network, ai_input, ai_output);

```

推理完成后，程序依次比较 10 个输出概率并找出最大值对应的数字：

```

for (uint32_t i = 0; i < AI_NETWORK_OUT_1_SIZE; i++)
{
    if (aiOutData[i] > max)
    {
        max = aiOutData[i];
        count = (int)i;
    }
}

```

5.1.5 推理结果输出

为了减少电脑端串口读取次数，程序将识别结果和各类别概率拼接到一个字符串缓冲区中，再一次性返回。

```
char sendBuf[256];
int offset = 0;

offset += sprintf(sendBuf + offset,
                  "current number is %d\r\n",
                  count);

offset += sprintf(sendBuf + offset, "P:");

for (uint32_t i = 0; i < AI_NETWORK_OUT_1_SIZE; i++)
{
    offset += sprintf(sendBuf + offset,
                      "%d=%.4f ",
                      (int)i,
                      aiOutData[i]);
}

offset += sprintf(sendBuf + offset, "\r\n");

HAL_UART_Transmit(&huart1,
                  (uint8_t *)sendBuf,
                  offset,
                  0xffff);
```

这种方式可以减少多次调用串口发送函数造成的显示延迟，并使上位机界面输出更加紧凑。

5.2 串口通信设计

5.2.1 串口基本配置

本项目使用 USART1 实现电脑端与 STM32 之间的通信。串口参数如下：

配置项	参数
串口外设	USART1
发送引脚	PA9
接收引脚	PA10
波特率	115200 bit/s
数据位	8 位
停止位	1 位
校验位	无

硬件流控	无
工作模式	同时支持发送与接收
接收方式	中断接收

5.2.2 数据帧格式

电脑端向 STM32 发送一帧手写图像数据。程序中定义：

```
#define UART_BUFF_LEN 1024
#define ONE_FRAME_LEN 1 + 784 + 2
```

即一帧数据共包含 787 字节，包括 1 字节帧头，784 字节图像像素数据及 2 字节结束字符。当 STM32 完整收到一帧数据后，可以在电脑端看到 rx length = 787，说明图像数据已成功传输。

5.2.3 中断接收

STM32 使用串口中断逐字节接收数据：

```
HAL_UART_Receive_IT(&huart1, (uint8_t *)&uart_rx_byte, 1);
```

在接收完成回调函数中将每个字节存入缓存数组：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *UartHandle)
{
    if (goRunning == 0)
    {
        if (uart_rx_length < UART_BUFF_LEN)
        {
            uart_rx_buffer[uart_rx_length] = uart_rx_byte;
            uart_rx_length++;

            if (uart_rx_byte == '\n')
            {
                goRunning = 1;
            }
        }
        else
        {
            uart_rx_length = 0;
        }
    }

    HAL_UART_Receive_IT(&huart1, (uint8_t *)&uart_rx_byte, 1);
}
```

```
if (goRunning > 0)
{
    if (uart_rx_length == ONE_FRAME_LEN)
    {
        PictureCharArrayToFloat(
            uart_rx_buffer + 1,
            aiInData,
            28 * 28
        );

        AI_Run(aiInData, aiOutData);
    }

    memset(uart_rx_buffer, 0, sizeof(uart_rx_buffer));
    goRunning = 0;
    uart_rx_length = 0;
}
```

5.3 上位机界面与数据传输

5.3.1 上位机界面及功能

由于开发板未配备触摸屏，为实现手写数字图像的输入与识别结果展示，本项目采用上位机程序 HandWriteApp 作为用户交互界面。该程序提供手写绘图区、串口选择、串口开关、波特率设置和数据发送等功能，可以实现电脑端与 STM32 开发板之间的数据通信。

用户首先在界面左侧区域绘制数字，随后选择对应串口并设置波特率。本项目使用的波特率为：

115200 bit/s 点击 Send 按钮后，上位机将手写图像转换为 28×28 像素数据，并通过串口发送至 STM32。STM32 接收到完整数据帧后，调用部署在芯片中的神经网络模型完成推理，再将识别结果和各类别概率返回电脑端。最终结果显示在界面右侧的文本框中。

5.3.2 串口读取方式优化

在初始版本中，上位机采用 ReadLine() 方法读取串口数据：

```
string msg = serialPort.ReadLine();
```


ReadLine() 会持续等待换行符。当串口关闭、读取线程退出或 I/O 操作被中止时，程序可能出现串口读取异常，并弹出错误提示。

为避免阻塞等待，后续将读取方式改为：

```
string msg = serialPort.ReadExisting();
```

优化后的接收函数如下：

```
void SerialReadEven(object sender, SerialDataReceivedEventArgs e)
{
    try
    {
        string msg = serialPort.ReadExisting();

        if (string.IsNullOrEmpty(msg))
        {
            return;
        }

        this.BeginInvoke(new Action(() =>
        {
            tb_log.AppendText(msg);
        }));
    }
    catch
    {
        // 串口关闭或线程结束时忽略异常
    }
}
```

ReadExisting() 会读取串口缓冲区中当前已经到达的数据，而不会持续等待换行符。与此同时，使用 AppendText() 将新数据追加至文本框，可以避免反复拼接原有文本。经过修改后，上位机能够更加稳定地接收 STM32 返回的识别结果和概率信息。

六、 系统部署与调试

6.1 STM32CubeMX 工程配置

首先在 STM32CubeMX 中创建基于 STM32F767 开发板的工程，根据系统需求配置晶振、时钟树、串口通信接口以及 STM32Cube.AI 中间件。串口用于实现开发板与上位机之间的数据传输，确保图像数据和识别结果能够可靠收发。

配置完成后，生成 STM32CubeIDE 工程文件，为后续模型部署与程序开发提供基础工程环境。

6.2 AI 模型导入与代码生成

在 STM32Cube.AI 中导入预训练模型 trained_lstm.tflite，并对模型进行解析与

验证。STM32Cube.AI 自动完成神经网络结构分析、存储资源评估以及代码生成工作，将 TensorFlow Lite 模型转换为适用于 STM32 平台运行的 C 语言神经网络推理代码。

生成内容包括模型参数、网络结构以及推理接口等文件，随后即可通过调用相应接口完成神经网络初始化与推理计算。

6.3 工程编译与程序烧录

在 STM32CubeIDE 中打开生成的工程，在 main.c 中编写串口通信、数据接收、图像预处理以及神经网络推理调用等代码，并完成整体系统集成。

随后利用 ST-Link 调试器将编译生成的程序下载至 STM32 开发板实现系统部署，烧录完成后开发板即能独立运行神经网络推理任务。

6.4 系统联调与运行验证

系统运行时，通过 USB 接口将板与计算机连接，实现串口通信。启动 HandWriteApp 上位机程序后用户可在手写区域输入数字，并通过串口将图像数据发送至 STM32 开发板。

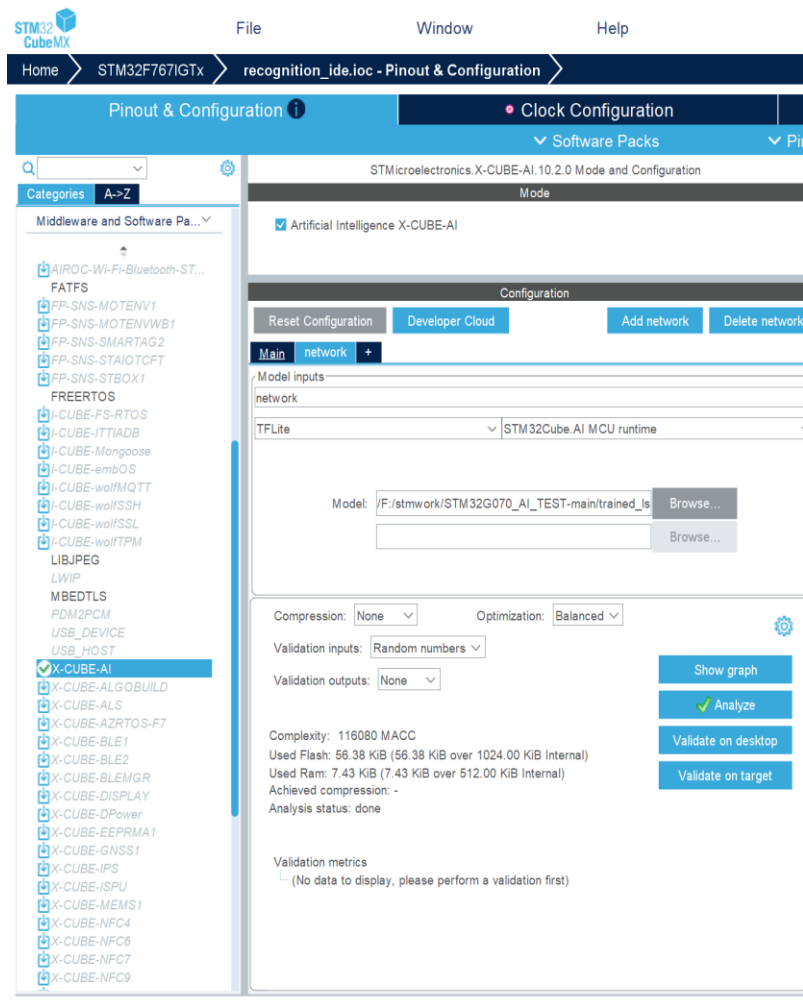
开发板接收数据后，调用部署在本地的神经网络模型进行推理计算，并将识别结果及对应概率返回至上位机。上位机接收到结果后，在界面中显示预测数字及各类别概率信息，从而完成手写数字识别的完整流程。

通过系统联调验证，开发板能够稳定完成图像接收、模型推理以及结果返回等功能，实现手写数字识别系统的预期设计目标。

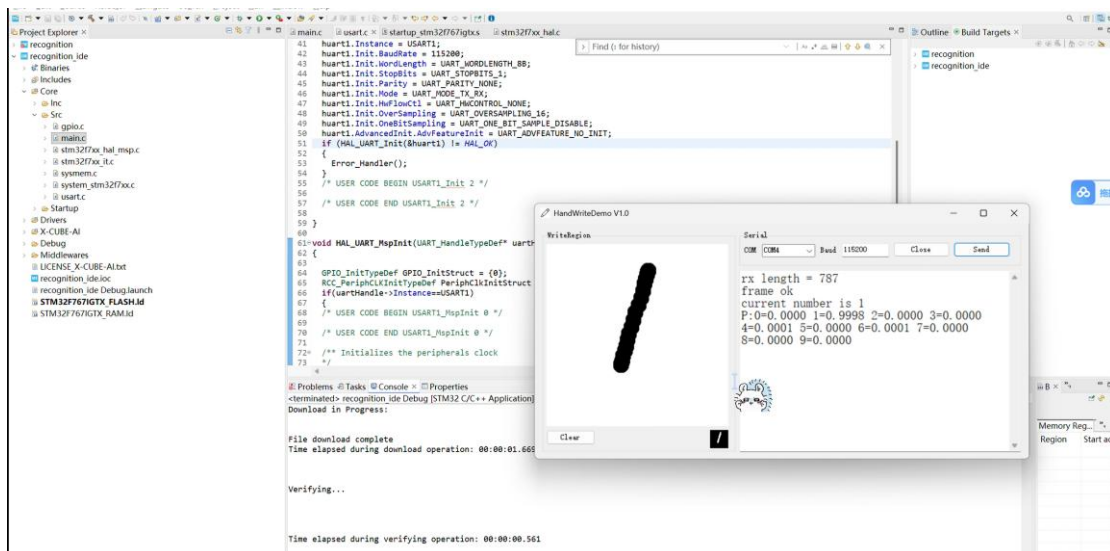
七、实验结果

7.1 实验截图

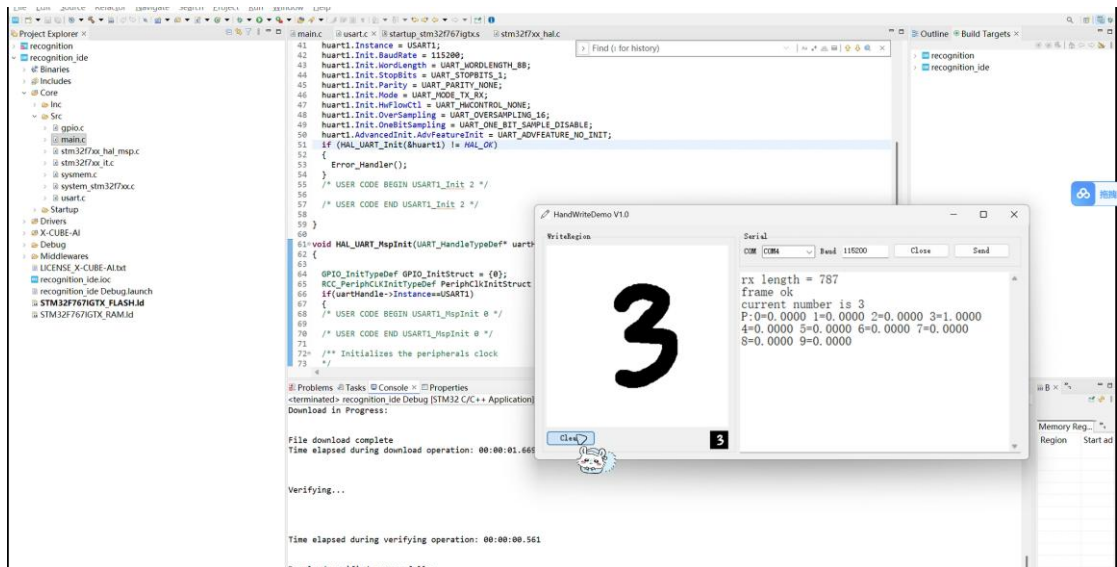
1. STM 界面截图：



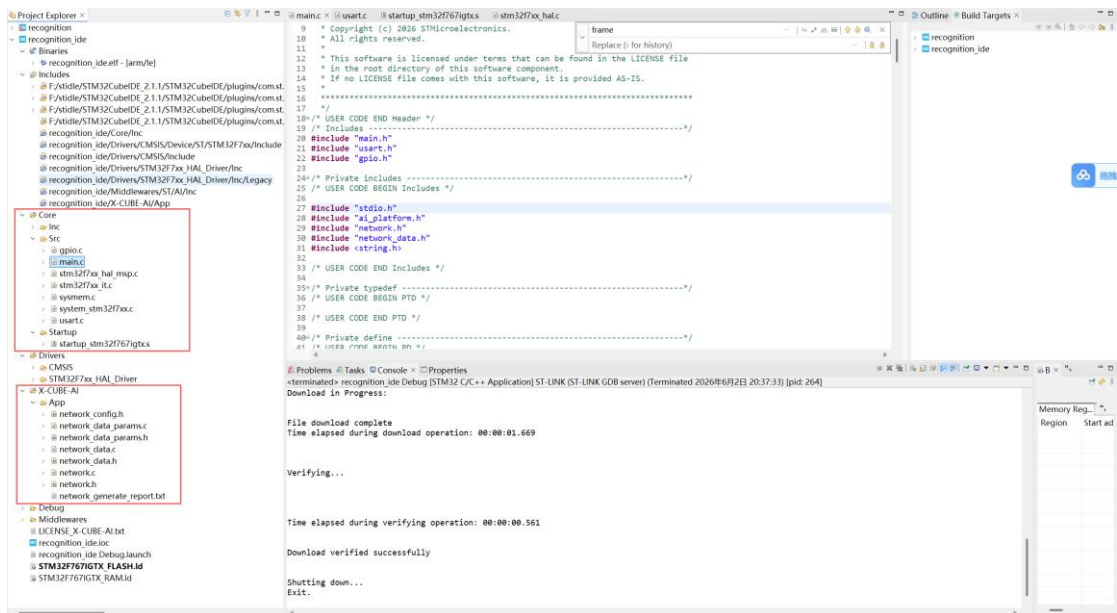
2. 绘制数字 1 后的识别结果截图；



3. 绘制数字 3 后的识别结果截图；



4. STM32CubeIDE 工程结构截图；



八、项目总结

8.1 调试过程中的困难及解决方法

1. 开发平台选取

本项目最初选择 keil 作为嵌入式开发平台，但初次编译程序后显示 keil 处于受限版本，生成的固件过大（约 65KB）而超过了当前链接器允许的容量（32KB）。因此更换至 CubeIDE 作为集成开发环境完成了工程开发、代码编译、下载及调试。

2. 串口未返回结果

最初程序启动后没有主动打印信息，难以判断串口是否正常工作。通过增加启动提示和回显功能实现对串口发送与接收功能的验证。

3. 分类结果显示不完整

程序最初将概率和分类结果逐行发送，当电脑端读取速度和刷新逻辑不稳定时最后一行可能无法及时显示。解决方法是优先发送最终分类结果，再发送各类别概率：

```
sprintf(logStr, "current number is %d\r\n", count);  
Uart_send(logStr);
```

4. 串口读取错误

电脑端使用 `ReadLine()` 时可能因为阻塞读取而出现：

Serial read error: 由于线程退出或应用程序请求，I/O 操作已中止。

改用 `ReadExisting()` 并忽略关闭串口时产生的异常后程序稳定性得到提高。

8.2 总结与展望

本项目完成了一个基于 STM32 和 X-CUBE-AI 的手写数字识别系统。系统能够接收电脑端发送的手写图像，在 STM32 中完成神经网络推理，并返回预测数字和各类别概率。

通过本项目的设计与实现，可以系统了解嵌入式人工智能系统的完整部署流程，包括模型导入与转换、STM32Cube.AI 代码生成、嵌入式平台部署、图像数据传输、边缘端推理以及识别结果展示等关键环节。本项目验证了在资源有限的微控制器中运行神经网络模型的可行性，也为进一步学习边缘人工智能和嵌入式视觉应用奠定了基础。

未来，项目可在以下方面继续优化：

1. 优化数据帧格式，使用固定帧头和固定长度数据，减少误判；
2. 增加校验位，提高串口通信可靠性；
3. 优化电脑端显示界面使结果更加直观；增加实时识别功能，减少手动点击发送的步骤；
4. 尝试部署更复杂的图像分类模型，并对比不同模型大小、推理时间和识别准确率之间的差异。